Project Overview

Our project aims to build a Retrieval-Augmented Generation (RAG) based LLM system that translates SQL queries from the SQLite to Apache HiveQL while being functionally equivalent.

This report includes identification, quality assessment, preprocessing, and structuring of our three primary data sources: the parallel query corpus, the schema contexts, and the official documentation for the RAG based generation.

Data Sources Identification & Verification

We relied on three distinct data sources to ensure structural accuracy, schema awareness, and syntax compliance.

Primary Dataset

Dataset Name: PARROT

Source: Hugging Face (weizhoudb/PARROT)

Key Details

- Available in Structured JSON/CSV format
- This dataset is publically available and it has no commercial restrictions
- We are filtering the full dataset exclusively for SQLite → HiveQL translation pairs and using it for our project

RAG Based Approach

To ground the LLM and prevent the hallucination of unsupported functions we utilise official documentation.

Apache Hive Language Manual

Used for RAG based generation covering DDL/DML syntax rules, built-in function mappings and type system references.

SQLite Core Language Manual

Used for RAG based generation covering DDL/DML syntax rules, built-in function mappings and type system references.

Data Models & Schemas

Translating SQL correctly requires context especially for dynamic to static typing.

Source

Extracted from the PARROT dataset environments and augmented with standard normalised database schemas.

Schema Elements

Mainly we are using table definitions, column data types and primary/foreign key relationships corresponding to the queries in the PARROT dataset.

Objective

By using these sources, we are trying to decrease hallucinations and generate functionality equivalent SQL queries in HiveQL from SQLite.

Dataset Description & Feature Distribution

After filtering the PARROT dataset to retain only the SQLite to HiveQL translation pairs, the resulting dataset contains a curated collection of parallel SQL queries used for training the translation model.

Dataset prepared for this project consists of:

Parallel Query Pairs

Approximately 1,000 to 2,000 high-quality parallel query pairs where each SQLite query has a corresponding HiveQL translation.

Unique DB Schemas

Around 50 to 100 unique database schemas are associated with these queries.

RAG Document Chunks

A corpus of 2,000 to 5,000 document chunks extracted from official database manuals and documentation.

SQL Feature Distribution

Low Complexity Queries

- Basic SELECT statements
- WHERE clauses
- LIMIT clause differences used for pagination

Medium Complexity Queries

- JOIN operations
- String manipulation functions
- SUBSTR (SQLite) → SUBSTRING (HiveQL)
- Common Table Expressions (CTEs)

High Complexity Queries

- Date and time function conversions
- strftime → from_unixtime
- Complex Window Functions
- Dynamic-to-static type casting differences

Dataset Quality Assessment

Additional quality checks are performed during preprocessing to ensure the dataset is clean and reliable for training.

Checks focus on detecting:

Missing Values

Identifying and handling absent data fields across query pairs.

Duplicate Entries

Removing repeated query pairs that could bias the training process.

Malformed SQL Queries

Detecting syntactically invalid or structurally broken SQL statements.

Encoding Inconsistencies

Resolving character encoding issues that may corrupt query data.

Schema Mismatches

Ensuring that only valid and meaningful query pairs remain in the dataset.

Dataset Adequacy & Augmentation Strategy

Foundation

The PARROT SQLite → Hive subset provides a strong foundation in terms of data quality.

However, the dataset size (~1,500 query pairs) sits near the lower bound typically recommended for reliable LLM fine-tuning.

Augmentation Pipeline

The augmentation process consists of three stages:

This pipeline allows us to expand the dataset while maintaining the same quality and semantic correctness guarantees as the original PARROT data.

01

Synthetic Query Generation

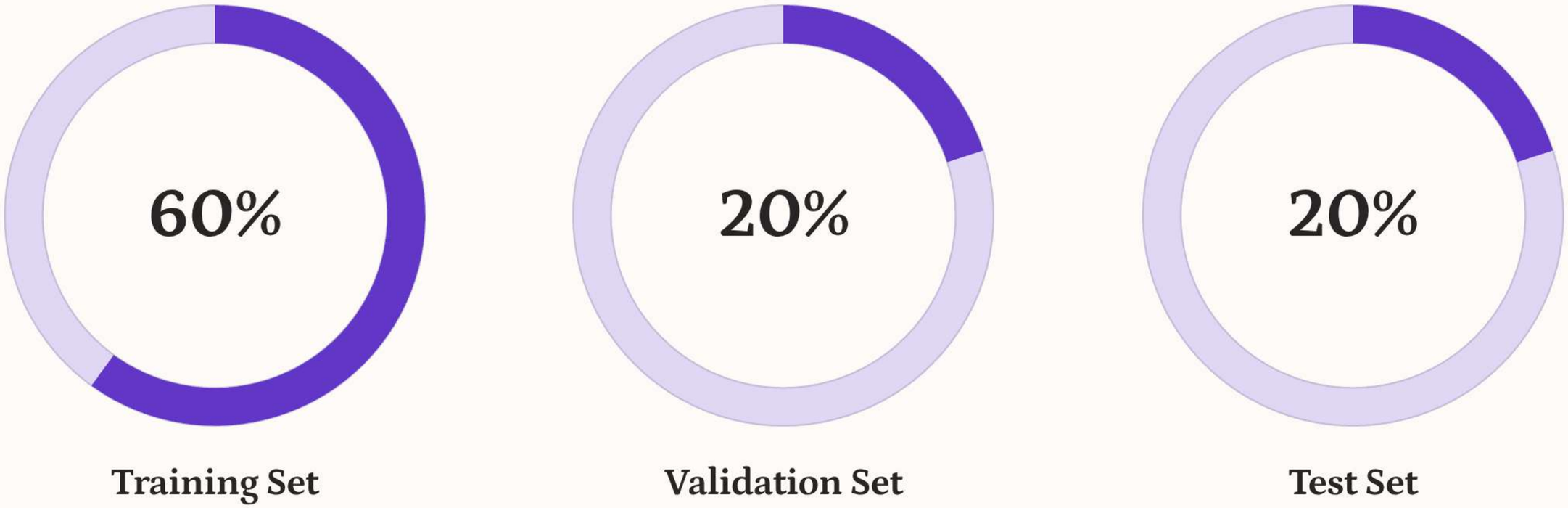
02

Deterministic Translation

03

Execution Validation

Dataset Split & LLM Formatting



Safeguards Against Information Leakage

Exact Deduplication	**Paraphrastic Similarity Detection**	**Schema-Level Isolation**

To fine-tune the Large Language Model (LLM), an Instruction–Response JSON structure is used to format the data. Including the database schema in the instruction helps the model generate more accurate translations and minimises ambiguity.

RAG Document Preparation

To enhance translation accuracy and minimise the generation of unsupported SQL functions, the system incorporates a Retrieval-Augmented Generation (RAG) component.

Chunking Strategy

Documentation is divided into 512-token chunks with a 64-token overlap.

Metadata Tagging

Metadata tags such as dialect, content type, and source are attached to each chunk.

Vector Embeddings

Text chunks are transformed into vector embeddings using a model like `text-embedding-3-small`.

Vector Store

ChromaDB is used during the development phase, transitioning to FAISS for production deployment.